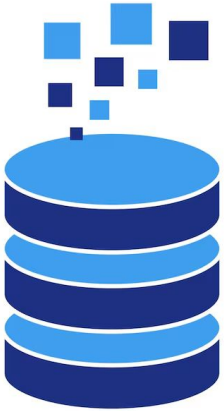
Fizičko projektovanje baze i optimizacija podataka u MS SQL bazi podataka

Anastasija Savov 1924



Kako su organizovani podaci u memoriji?

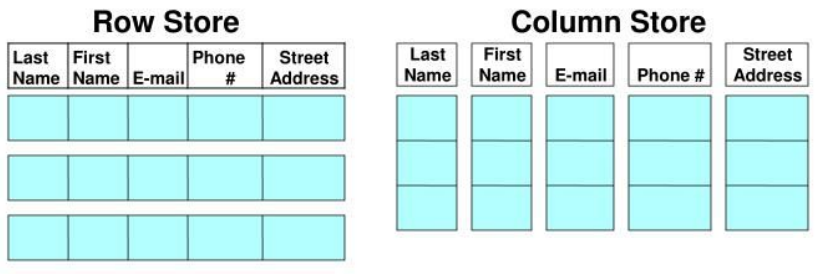
- **Stranice** - osnovna jedinica skladištenja IO operacija, fiksna veličina 8KB (96B zauzima header); mogu biti index, data, LOB, GAM, IAM, PFS stranice
- **Extent-i** (grupa 8 uzastopnih stranica): mogu biti **uniformni** i **razmešani**

Results | Messages

	TableName	RowCount	TotalPages	TotalSpaceMB	UsedPages	DataPages
1	LogPoruke	100000	100665	786	100646	100645
2	Arhiva	10000	2182	17	2060	2034
3	Prodaja_RowStore	100010	489	3	483	481
4	Prodaja_ColumnStore	100010	9	0	2	0

Koje su vrste skladišta?

- Po redovima (**row-store**)
  - Prednosti:
    - Efikasne OLTP operacije
    - INSERT, UPDATE, DELETE
  - Mane:
    - Ograničena kompresija
    - Neefikasna za analitičke upite
- Po kolonama (**column-store**)
  - Prednosti:
    - Batch processing
  - Mane:
    - Delta store overhead



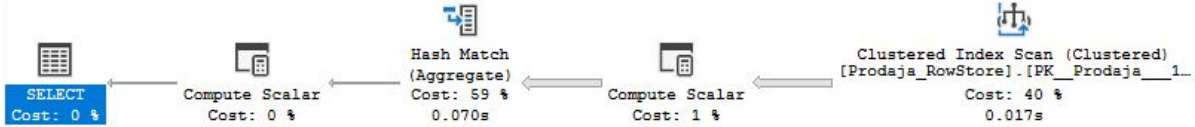
```sql
SELECT
    Region,
    YEAR(DatumProdaje) AS Godina,
    SUM(Kolicina * Cena) AS UkupnaProdaja
FROM Prodaja_ColumnStore
GROUP BY Region, YEAR(DatumProdaje);
```

107 %

Results | Messages | Live Query Statistics | Execution plan

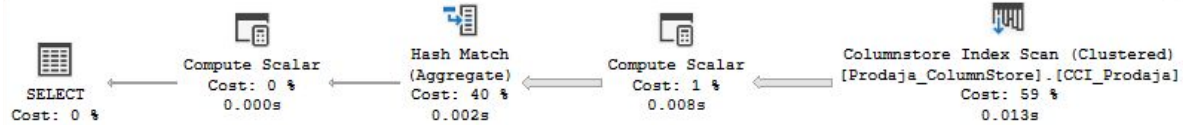
Query 1: Query cost (relative to the batch): 87%

SELECT Region, YEAR(DatumProdaje) AS Godina, SUM(Kolicina * Cena) AS UkupnaProdaja FROM Prodaja_RowStore GROUP BY Region, YEAR(DatumProdaje)

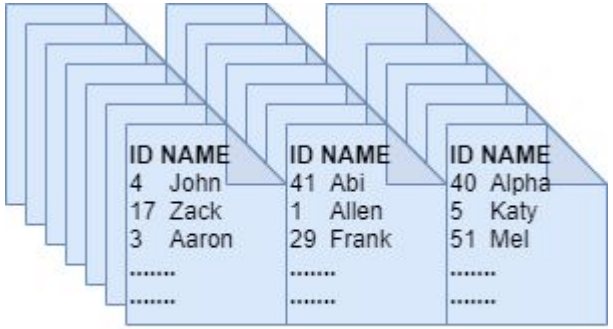


Query 2: Query cost (relative to the batch): 13%

SELECT Region, YEAR(DatumProdaje) AS Godina, SUM(Kolicina * Cena) AS UkupnaProdaja FROM Prodaja_ColumnStore GROUP BY Region, YEAR(DatumProdaje)



Heap i klasterovane tabele

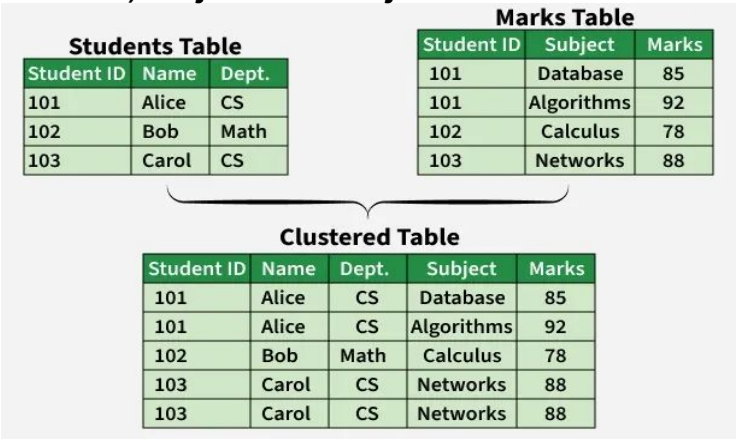


Heap tabela

- tabela bez klasterovanog indeksa
- Redovi se dodaju na prvu dostupnu lokaciju bez sortiranja
- **Forwarding pointer** - kada se red poveća i ne stane na originalnu stranicu, kreira se pokazivač na novu lokaciju
- Lokacija svakog reda se identikuje putem **RID** (Row Identier) koji se sastoji od FileID:PageID:SlotID

Klasterovana tabela

- Nema forwarding pointer problema
- Koriste klasterovani ključ kao pokazivač
- **Range scan** je veoma ekasan



Students Table

Student ID	Name	Dept.
101	Alice	CS
102	Bob	Math
103	Carol	CS

Marks Table

Student ID	Subject	Marks
101	Database	85
101	Algorithms	92
102	Calculus	78
103	Networks	88

Clustered Table

Student ID	Name	Dept.	Subject	Marks
101	Alice	CS	Database	85
101	Alice	CS	Algorithms	92
102	Bob	Math	Calculus	78
103	Carol	CS	Networks	88
103	Carol	CS	Networks	88

Problem forwarding pointer-a kod heap tabele

```sql
-- Kreiranje heap tabele (bez PRIMARY KEY CLUSTERED)
CREATE TABLE LogPoruke (
    LogID INT IDENTITY,
    Poruka VARCHAR(MAX),
    DatumVreme DATETIME2 DEFAULT SYSDATETIME()
);

-- Provera forwarding pointer-a
SELECT
    OBJECT_NAME(object_id) AS TableName,
    forwarded_record_count,
    page_count
FROM sys.dm_db_index_physical_stats(
    DB_ID(), OBJECT_ID('LogPoruke'), 0, NULL, 'DETAILED')
WHERE index_id = 0;  -- 0 = heap
```

Results | Messages

	TableName	forwarded_record_count	page_count
1	LogPoruke	99999	100645

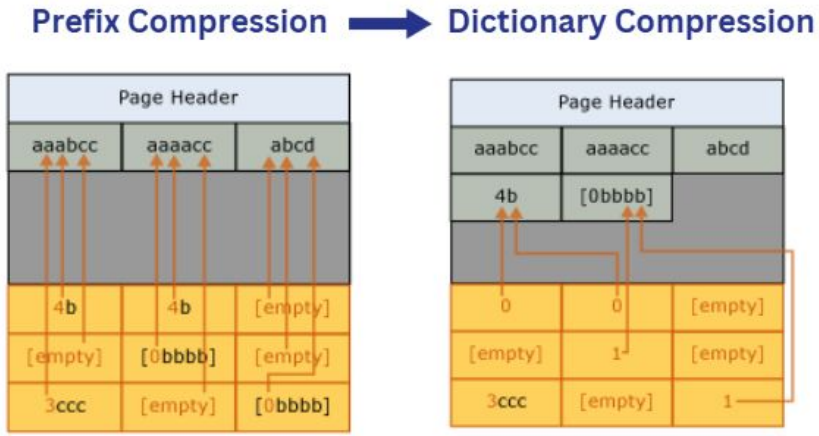
Kompresija podataka

ROW kompresija

- Fiksni tipovi podataka se čuvaju u formatu promenljive dužine
- NULL i neke numeričke vrednosti (npr. 0) mogu zauzimati 0 bajta
- svaki red se kompresuje nezavisno

PAGE kompresija

- Prefix kompresija: zajednički prefiksi se izdvajaju i referenciraju
- Dictionary kompresija: ponavljajući obrasci na stranici se zamenjuju kraćim referencama
- Veća kompresija + veći CPU overhead



Results | Messages

	TableName	RowCounts	TotalSpaceKB	UsedSpaceKB	UnusedSpaceKB
1	LogPoruke	100000	805320	805168	152
2	Arhiva	10000	37896	37864	32
3	Prodaja_RowStore	100010	3912	3864	48
4	Prodaja_ColumnStore	100010	72	16	56

Pre kompresije

ROW kompresija

Results | Messages

	TableName	RowCounts	TotalSpaceKB	UsedSpaceKB	UnusedSpaceKB
1	LogPoruke	100000	805320	805168	152
2	Arhiva	10000	17456	16480	976
3	Prodaja_RowStore	100010	3912	3864	48
4	Prodaja_ColumnStore	100010	72	16	56

	TableName	RowCounts	TotalSpaceKB	UsedSpaceKB	UnusedSpaceKB
1	LogPoruke	100000	805320	805168	152
2	Prodaja_RowStore	100010	3912	3864	48
3	Arhiva	10000	1008	416	592
4	Prodaja_ColumnStore	100010	72	16	56

PAGE kompresija

Radno opterećenje (workload)

Skup upita i UPDATE operacija koje sistem treba da podrži

Za svaki upit potrebno je odrediti:

- Kojim se relacijama (tabelama) pristupa
- Koji se atributi traže u **SELECT** iskazu
- Nad kojim atributima se vrši spajanje ili selekcija u **WHERE** iskazu i koliko su ti uslovi selektivni
- Koliko se **često** upit izvršava (učestalost) i da li je vremenski kritičan

Radno opterećenje - odluke

Na osnovu analize odluke koje treba da se donesu su:

1. Koje indekse napraviti - Koje relacije indeksirati, koja polja izabrati za klju£ pretrage, da li indeks treba biti klasterovan ili ne, redak ili gust

2. Da li menjati konceptualnu shemu - Razmotriti denormalizaciju, vertikalno particionisanje ili uvođenje pogleda

3. Da li preformulisati upite - često izvršavani upiti mogu biti preformulisani tako da budu efikasniji

Projektovanje primarnog i stranog ključa

```sql
-- Surogat ključ
CREATE TABLE Kupci
(
    KupacID INT IDENTITY NOT NULL,
    JMBG CHAR(13) NOT NULL,
    Ime NVARCHAR(50) NOT NULL,
    Prezime NVARCHAR(50) NOT NULL,
    Email VARCHAR(100),
    CONSTRAINT PK_Kupci PRIMARY KEY CLUSTERED (KupacID),
    CONSTRAINT UQ_Kupci_JMBG UNIQUE (JMBG)
);

-- Kompozitni ključ u asocijativnoj tabeli
CREATE TABLE KupacProizvod
(
    KupacID INT NOT NULL,
    ProizvodID INT NOT NULL,
    DatumKupovine DATE NOT NULL DEFAULT GETDATE(),
    CONSTRAINT PK_KupacProizvod
        PRIMARY KEY CLUSTERED (KupacID, ProizvodID),
    CONSTRAINT FK_KP_Kupac
        FOREIGN KEY (KupacID) REFERENCES Kupci(KupacID),
    CONSTRAINT FK_KP_Proizvod
        FOREIGN KEY (ProizvodID) REFERENCES Proizvodi(ProizvodID)
);
```

```sql
-- Funkcija nad kolonom, cesto vodi ka skeniranju indeksa
SELECT NarudzbinaID, KupacID, DatumNarudzbine
FROM Narudzbine
WHERE YEAR(DatumNarudzbine) = 2025;

-- Bolje resenje: opseg nad kolonom, pogodnije za Index Seek
SELECT NarudzbinaID, KupacID, DatumNarudzbine
FROM Narudzbine
WHERE DatumNarudzbine >= '2025-01-01'
    AND DatumNarudzbine <  '2026-01-01';
```

Range filter umesto pozivanja funkcije nad kolonom

Selektovanje odredjenih kolona umesto svih

```sql
-- Neoptimalan upit: select svih kolona
SELECT *
FROM IstorijaNarudzbina
WHERE KupacID = @KupacID;

-- Optimalniji upit: select samo potrebnih kolona
SELECT NarudzbinaID, DatumNarudzbine, Iznos
FROM IstorijaNarudzbina
WHERE KupacID = @KupacID;
```

```sql
-- Filtriranje nakon join-a i potencijalna implicitna konverzija
SELECT n.NarudzbinaID, k.Ime, k.Prezime
FROM Narudzbine n
JOIN Kupci k ON CONVERT(VARCHAR(20), k.KupacID) = n.KupacIDTekst
WHERE n.Status = 'Aktivna';

-- Poboljsanje: uskladjeni tipovi i direktan JOIN uslov
SELECT n.NarudzbinaID, k.Ime, k.Prezime
FROM Narudzbine n
JOIN Kupci k ON k.KupacID = n.KupacID
WHERE n.Status = 'Aktivna';
```

Uklanjanje konverzije - uskladjivanje tipova

Izbor tipova podataka

```sql
-- LOŠ izbor tipova podataka
CREATE TABLE Zaposleni_Los
(
    ZaposleniID BIGINT IDENTITY,          -- prevelik tip
    Ime NVARCHAR(MAX),                    -- nepotrebno MAX
    Starost FLOAT,                        -- neprecizan za cele brojeve
    Plata FLOAT,                          -- greške zaokruživanja
    DatumZaposlenja DATETIME,             -- nepotrebna preciznost
    Aktivan NVARCHAR(10)                  -- string umesto bita
);

-- DOBAR izbor tipova podataka
CREATE TABLE Zaposleni_Dobar
(
    ZaposleniID INT IDENTITY,             -- dovoljan opseg
    Ime NVARCHAR(100),                    -- ograničena dužina
    Starost TINYINT,                      -- 0-255 je dovoljan
    Plata DECIMAL(12,2),                  -- precizan tip
    DatumZaposlenja DATE,                 -- samo datum
    Aktivan BIT DEFAULT 1                -- logička vrednost
);
```

Ograničenja

Definisati kolone kao **NOT NULL** gde god je moguće

CHECK ograničenja - Ograničavaju dozvoljene vrednosti kolone. Optimizator može koristiti CHECK ograničenja za eliminaciju nepotrebnih particija ili preskakanje delova podataka

DEFAULT vrednosti - Obezbeđuju da kolone imaju smislenu početnu vrednost, smanjujući pojavu NULL vrednosti i pojednostavljujući INSERT operacije

UNIQUE ograničenja - Garantuju jedinstvenost vrednosti i implicitno kreiraju neklasterovani indeks, koji se može koristiti za brzo pretraživanje

Projektovanje tabela i relacija

Particionisanje tabela

RANGE particionisanje

HASH i **LIST** particionisanje nisu podržani u SQL serveru

```sql
-- 1. Kreiranje particione funkcije
CREATE PARTITION FUNCTION PF_PoGodini (DATE)
AS RANGE RIGHT FOR VALUES
('2023-01-01', '2024-01-01', '2025-01-01');

-- 2. Kreiranje particione sheme
CREATE PARTITION SCHEME PS_PoGodini
AS PARTITION PF_PoGodini
ALL TO ([PRIMARY]);

-- 3. Kreiranje particionisane tabele
CREATE TABLE IstorijaNarudzbina
(
    NarudzbinaID INT IDENTITY NOT NULL,
    DatumNarudzbine DATE NOT NULL,
    KupacID INT NOT NULL,
    Iznos DECIMAL(12,2),
    CONSTRAINT PK_IstorijaNarudzbina
        PRIMARY KEY CLUSTERED (NarudzbinaID, DatumNarudzbine)
) ON PS_PoGodini(DatumNarudzbine);
```

107 %

Results | Messages

	BrojParticije	GranicaParticije	BrojRedova
1	1	NULL	0
2	2	2023-01-01 00:00:00.000	3345
3	3	2024-01-01 00:00:00.000	8874
4	4	2025-01-01 00:00:00.000	8885

Preporuke

- Koristiti najmanje moguće tipove podataka
- Koristiti ograničenja (NOT NULL, CHECK, DEFAULT, UNIQUE)
- Particionisati velike tabele
- Izbor normalne forme
- Izbor ključeva i indeksa

Projektovanje indeksa

- najznačajniji mehanizam za optimizaciju performansi upita
- B+ tree struktura
- **Klasterovani indeksi** (uzak, jedinstven, statičan, monotono rastući)
- **Neklasterizovani indeksi**
- Indeksi koji poboljšavaju performanse
  - **Kompozitni** (indeksira se više kolona, redosled je jako važan)
  - **Filtrirani** (indeksira se samo podskup redova)
  - **Pokrivajući** (covering - sadrži sve kolone potrebne za upit)

Preporuke

- Ne praviti indeks ako ne ubrzava nijedan upit
- Atributi iz **WHERE** iskaza su primarni kandidati za indeksiranje
- Razmotriti uticaj indeksa na **INSERT/UPDATE/DELETE**; odbaciti indeks ako previše usporava ažuriranja (ali imati na umu da indeksi mogu i ubrzati ažuriranja pronalaženjem redova)
- **Koklasterovanje** (Fizički smestiti redove iz povezanih tabela blizu na disku - manje I/O pri JOIN operacijama)
    - Zajednička filegroup za povezane tabele
    - Indeksirani pogledi (materijalizacija JOIN rezultata)

Obrada transakcija, planovi izvršavanja, izolacija i zaključavanje u MongoDB bazi podataka

Anastasija Savov, 1924

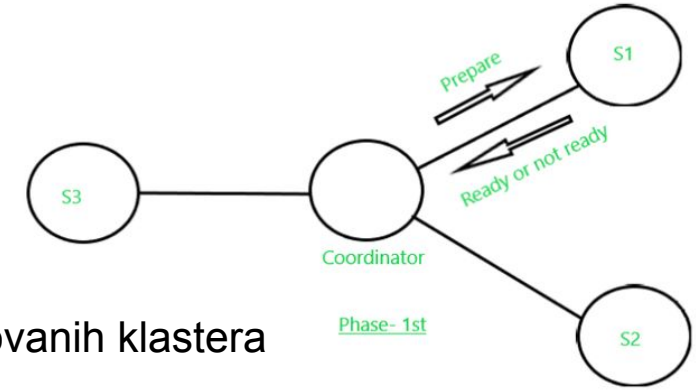
Model podataka i atomičnost

- Sve operacije nad jednim dokumentom su **atomične** (ugnježdeni dokumenti, nizovi)
- Preporučene strategije: **ugnježdavanje** i **denormalizacija** → smanjuje potrebu za transakcijama
- Kada dokument prelazi preko organičenja od **16MB** ili postoje n:m veze - neophodne su **višedokumentne transakcije**

Višedokumentne ACID transakcije

- ACID princip: Atomičnost, Konzistentnost, Izolacija, Trajnost
- Transakcije rade kroz sesiju (**startSession** → **withTransaction**)
- Callback API (**withTransaction**) - automatski retry kod prolaznih grešaka
- Mane i ograničenja
  - **timeout 60s**,
  - maks. **16MB** oplog zapisa
  - nema savepoint-a
  - Write conflict (ako dve transakcije pokušaju da modifikuju isti dokument, druga transakcija će dobiti write conflict grešku i moraće da se ponovi)

Višedokumentne ACID transakcije



Distribuirane transakcije

- Transakcije koje obuhvataju više shardova, tj shardovanih klastera
- **2PC protokol (two-phase commit)**
- Transaction **Coordinator** - Jedan od shard-ova (obično onaj na kom je pokrenuta prva operacija pisanja) preuzima ulogu koordinatora. Koordinator prati učesnike transakcije i upravlja 2PC protokolom
- **Prepare faza** - Koordinator šalje komandu svakom učesničkom shard-u. Svaki učesnik priprema transakciju, drži lock-ove nad modifikovanim dokumentima i odgovara pozitivno ili negativno
- **Commit/Abort faza** - Ako su svi učesnici odgovorili pozitivno, koordinator šalje komandu za commit svim učesnicima. U slučaju bilo koje negativne potvrde ili isteka vremena, šalje se abort

Planovi izvršavanja upita

- **Cost-based optimizer** - testira više planova paralelno, kešira najbolji plan u tom momentu
- **explain()** funkcija - alat za analizu planova izvršavanja u MongoDB-u
- Ključne faze tokom izvršavanja upita: **COLLSCAN, IXSCAN, FETCH**, **SORT**
- Metrike: odnos **totalDocsExamined** i **nReturned** treba da bude približan 1
- Pipeline optimizacije: **predicate pushdown**, **sort-merge**, **projection pushdown**

COLLSCAN

IXSCAN

Primer za COLLSCAN

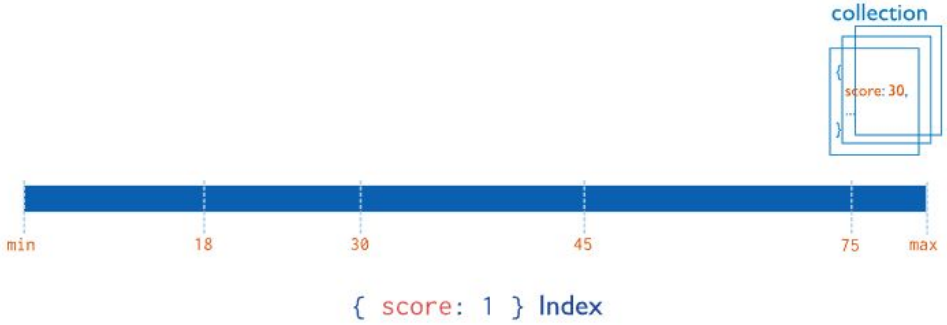
```
executionStats: {
  executionSuccess: true,
  nReturned: 0,
  executionTimeMillis: 0,
  totalKeysExamined: 0,
  totalDocsExamined: 5,
  executionStages: {
    isCached: false,
    stage: 'COLLSCAN',
    filter: { status: { '$eq': ' aktivna ' } },
    nReturned: 0,
    executionTimeMillisEstimate: 0,
    works: 6,
    advanced: 0,
    needTime: 5,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    direction: 'forward',
    docsExamined: 5
  }
},
```

Primer za IXSCAN

```
executionStats: {
  executionSuccess: true,
  nReturned: 2,
  executionTimeMillis: 8,
  totalKeysExamined: 2,
  totalDocsExamined: 2,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 2,
    executionTimeMillisEstimate: 0,
    works: 3,
    advanced: 2,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 2,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 2,
      executionTimeMillisEstimate: 0,
      works: 3,
      advanced: 2,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
```

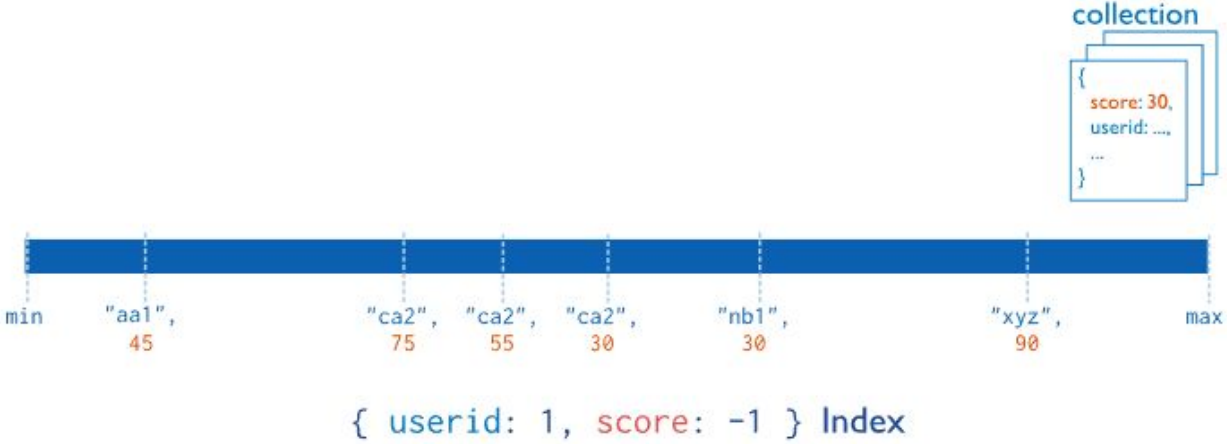
Indeksi i optimizacija

- Tipovi indeksa: **jednostruki, kompozitni, multikey, tekstualni, geospacijalni, hashed, wildcard**
- **ESR** pravilo za upotrebu kompozitnih indeksa: **Equality** → **Sort** → **Range**
- Covered query (koristi samo IXSCAN, bez FETCH) - najbrža varijanta pretrage

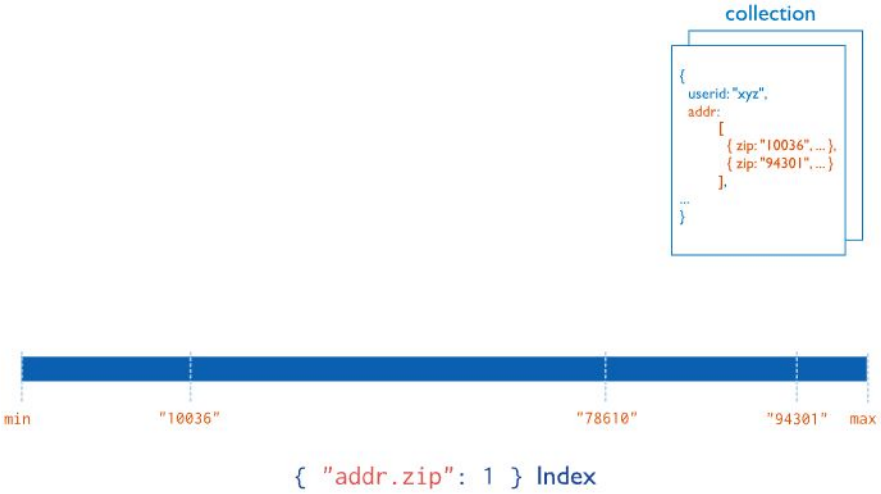


Jednostruki indeks

Kompozitni indeks



Multi-key indeks



Read i Write concern

- **Write Concern**
  - definiše nivo garancije da je operacija pisanja uspešno zabeležena
  - **w:0** (bez potvrde) → **w:1** (primarni) → **w:"majority"** (trajno) + **j:true** (journal)
- **Read Concern**
  - definiše nivo konzistentnosti i izolacije podataka koji se čitaju
  - **local** → **majority** → **linearizable** → **snapshot** (u transakcijama)
- **Kauzalna konzistentnost** - sesija garantuje ispravni redosled operacija i pri čitanju sa sekundarnih čvorova
  - Oslanja se na klaster logički sat za sinhronizaciju

Izolacija transakcija

- MongoDB koristi **snapshot** izolaciju
- Sprečava: dirty read, non-repeatable read, phantom read
- Ne sprečava **write skew** - dve transakcije čitaju iste podatke i na osnovu pročitanog donose odluke o pisanju koje su međusobno konfliktne
- **Write conflict** - prva transakcija koja ažurira dokument ga i zaključava, druga dobija WriteConflict grešku i mora da se ponovi
- Atomični operatori ($inc, $push) štite od **lost updates** van transakcija (kada dve konkurentne operacije međusobno prepisuju ažuriranja)

Simulacija WriteConflict greške

```
// Transakcija 1: čita i ažurira dokument
const session1 = db.getSession().getClient().startSession();
const session2 = db.getSession().getClient().startSession();

session1.startTransaction();
db.proizvodi.updateOne(
  { _id: "P001" },
  { $inc: { stanje: -5 } },
  { session: session1 }
);
// Transakcija 1 još nije potvrdila...

// Transakcija 2: pokušava da ažurira isti dokument
session2.startTransaction();
db.proizvodi.updateOne(
  { _id: "P001" },
  { $inc: { stanje: -3 } },
  { session: session2 }
);
// Rezultat: MongoServerError: WriteConflict error: this operation
   conflicted  with another operation. Please retry your operation or
   multi-document transaction.
```

Primer potencijalne lost updates greške

```
// Los primer: read-write van transakcije
// Ako dve instance izvrše ovo paralelno, jedna izmena se gubi
const p = db.proizvodi.findOne({ _id: "P001" });
db.proizvodi.updateOne(
  { _id: "P001" },
  { $set: { stanje: p.stanje - 5 } }  // <-- lost update rizik
);

// Dobar primer: atomični $inc - server izvršava inkrementiranje
// bez read-write, garantovano bez lost update-a
db.proizvodi.updateOne(
  { _id: "P001" },
  { $inc: { stanje: -5 } }
);

// Dodatna zaštita: uslovno ažuriranje samo ako ima dovoljno novca na
   stanju  (optimistička provera)
db.proizvodi.updateOne(
  { _id: "P001", stanje: { $gte: 5 } },
  { $inc: { stanje: -5 } }
);
```

Zaključavanje

- **WiredTiger** (od v3.2) - zaključavanje na nivou dokumenta
- **Hijerarhija**: Global → Database → Collection → Document
- Tipovi: **Shared** (S), **Exclusive** (X), **Intent Shared** (IS), **Intent Exclusive (**IX)
- **MVCC (multi-version concurrency control)**
  - *Snapshot* čitanje
  - In-memory upravljanje verzijama dokumenata
  - *Checkpoint* mehanizam
  - Oslobađanje starih verzija
- **Yielding** - dugotrajne operacije periodično oslobađaju lock-ove
- Proces ekvivalentan Strict 2PL: lock-ovi drže se do commit-a
- **Deadlock** procedura
  - **timeout** (5ms)
  - **WriteConflict**
  - **retry**

```
session.startTransaction();

// Growing faza: svaka operacija pisanja zaključava dokument
// i drži lock do kraja transakcije
db.racuni.updateOne(
  { _id: "racun_A" }, { $inc: { stanje: -1000 } }, { session }
);  // <- lock na racun_A se drži

db.racuni.updateOne(
  { _id: "racun_B" }, { $inc: { stanje: 1000 } }, { session }
);  // <- lock na racun_B se drži

// Svi lock-ovi se drže do ovog trenutka
```

```
// Paralelni čitaoci vide staro stanje (MVCC) - ne blokiraju se
session.commitTransaction();
// Shrinking faza: svi lock-ovi se oslobađaju istovremeno
```

Primer dvofaznog zaključavanja

Upravljanje vrućim tačkama i preporuke za optimizaciju

Upravljanje vrućim tačkama

- Kratke transakcije - manje konflikata
- Vremensko razdvajanje batch operacija - umesto konkurentnih transakcija nad istim dokumentima, koristiti red čekanja (queue pattern)
- **Dekompozicija** hotspot dokumenata
- Atomični operatori van transakcija - Kada je moguće, koristiti *$inc*, *$push*

Preporuke za optimizaciju

- Pravilna upotreba **Read/Write Concern-a** po tipu podataka
  - Za nekritične podatke (logovi, keš) - koristiti **w: 1** i readConcern: "**local**" za maksimalne performanse
  - Za poslovne podatke - koristiti **w: "majority"** i readConcern**: "majority"** za balans između performansi i trajnosti
  - Za kritične finansijske podatke - koristiti transakcije sa readConcern**: "snapshot"** i writeConcern**: w: "majority", j: true**}}
- Strategije za smanjenje write conflict-a (izbegavanje hotspotova, indeksiranje, retry, modelovanje podataka)

```
// Korišćenje withTransaction za automatski retry
function transferSredstava(saRacuna, naRacun, iznos) {
  const session = db.getMongo().startSession();

  try {
    session.withTransaction(function() {
      const racuni = session.getDatabase("banka").racuni;

      // Provera stanja pre transfera
      const izvor = racuni.findOne(
        { _id: saRacuna },
        { session }
      );

      if (izvor.stanje < iznos) {
        throw new Error("Nedovoljno sredstava");
      }

      // Atomični transfer
      racuni.updateOne(
        { _id: saRacuna },
        { $inc: { stanje: -iznos } },
        { session }
      );

      racuni.updateOne(
        { _id: naRacun },
        { $inc: { stanje: iznos } },
        { session }
      );
    }, {
      readConcern: { level: "snapshot" },
      writeConcern: { w: "majority" }
    });

    print("Transfer uspešan.");
  } finally {
    session.endSession();
  }
}
```

High Availability rešenja u MongoDB bazi podataka

Anastasija Savov, 1924

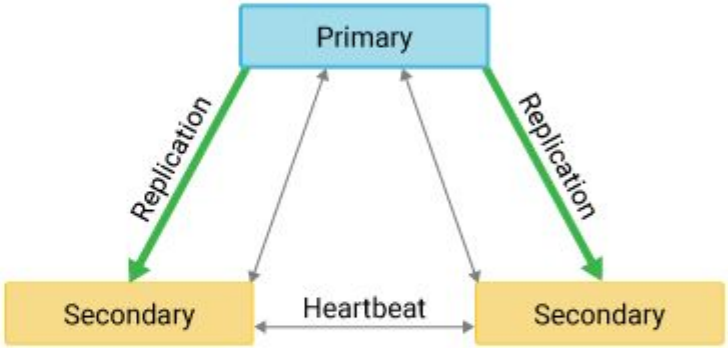
Uvod

- Visoka dostupnost = sposobnost sistema da radi bez prekida čak i pri kvarovima. Ključne metrike:
- **RTO** - maksimalno prihvatljivo vreme oporavka
- **RPO** - maksimalni prihvatljivi gubitak podataka
- **MTTR / MTBF** - prosečno vreme oporavka / između kvarova
- MongoDB postiže HA kroz: **replica sets**, **sharding**, automatski **failover** i konfigurabilne mehanizme konzistentnosti

Osnove visoke dostupnosti

- **"Broj devetki": 99,9%** = 8,7h prekida/god; **99,999%** = 5,25 min/god
- Eliminacija **SPOF** (Single Points of Failure) kroz **redundantnost podataka**
- **CAP teorema**: MongoDB bira CP (Consistency + Partition Tolerance), ali nudi Read/Write Concern za balans izmedju konzistentnosti i dostupnosti
- **Replikacija**: povećava dostupnost + distribuira opterećenje čitanja

Šta su replica setovi?



- Osnovna jedinica: skup **mongod** instanci koje čuvaju iste podatke
- Tipovi članova:
- **Primarni** - jedini prima write operacije
- **Sekundarni** - replicira oplog
- **Arbitar** - samo glasa, ne čuva podatke (jeftina redundantnost)
- **Oplog** - **WAL (write-ahead log)** kog sekundarni čvorovi prate asinhrono; idempotentnost
- Minimum za produkcijsko okruženje: **3 člana** (2 punopravna + arbitar, ili 3 punopravna)
- Parametri: priority, votes, hidden, secondaryDelaySecs, tags

Neuspelo dodavanje arbitra

```
}
rs0 [direct: primary] test> rs.addArb("localhost:27021")
MongoServerError[NodeNotFound]: Quorum check failed because not enough voting nodes responded; required 2 but only the fol
lowing 1 voting nodes responded: 127.0.0.1:27017; the following nodes did not respond affirmatively: localhost:27020 faile
d with Error connecting to localhost:27020 (127.0.0.1:27020) :: caused by :: onInvoke :: caused by :: No connection could
be made because the target machine actively refused it., localhost:27021 failed with Error connecting to localhost:27021 (
127.0.0.1:27021) :: caused by :: onInvoke :: caused by :: No connection could be made because the target machine actively
refused it.
```

Pokretanje sekundarnog čvora

```
  },
  {
    _id: 2,
    name: 'localhost:27018',
    health: 1,
    state: 5,
    stateStr: 'STARTUP2',
    uptime: 6,
    optime: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
    optimeDurable: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
    optimeWritten: { ts: Timestamp({ t:
    optimeDate: ISODate('1970-01-01T00:0
    optimeDurableDate: ISODate('1970-01-
    optimeWrittenDate: ISODate('1970-01-
    lastAppliedWallTime: ISODate('1970-0
    lastDurableWallTime: ISODate('1970-0
    lastWrittenWallTime: ISODate('1970-0
    lastHeartbeat: ISODate('2026-04-19T1
    lastHeartbeatRecv: ISODate('2026-04-
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: '127.0.0.1:27017',
    syncSourceId: 0,
    infoMessage: '',
    configVersion: 3,
    configTerm: 3
  }
],
```

```
  },
  {
    _id: 2,
    name: 'localhost:27018',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 112,
    optime: { ts: Timestamp({ t: 1776625620, i: 3 }), t: Long('3') },
    optimeDurable: { ts: Timestamp({ t: 1776625620, i: 3 }), t: Long('3') },
    optimeWritten: { ts: Timestamp({ t: 1776625620, i: 3 }), t: Long('3') },
    optimeDate: ISODate('2026-04-19T19:07:00.000Z'),
    optimeDurableDate: ISODate('2026-04-19T19:07:00.000Z'),
    optimeWrittenDate: ISODate('2026-04-19T19:07:00.000Z'),
    lastAppliedWallTime: ISODate('2026-04-19T19:07:00.205Z'),
    lastDurableWallTime: ISODate('2026-04-19T19:07:00.205Z'),
    lastWrittenWallTime: ISODate('2026-04-19T19:07:00.205Z'),
    lastHeartbeat: ISODate('2026-04-19T19:07:05.310Z'),
    lastHeartbeatRecv: ISODate('2026-04-19T19:07:05.960Z'),
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: '127.0.0.1:27017',
    syncSourceId: 0,
    infoMessage: '',
    configVersion: 4,
    configTerm: 3
```

```
mongod --replSet "rs0" --port 27018 --dbpath /data/db2 --bind_ip
    localhost

// Dodavanje novog sekundarnog čvora
rs.add("localhost:27018")
```

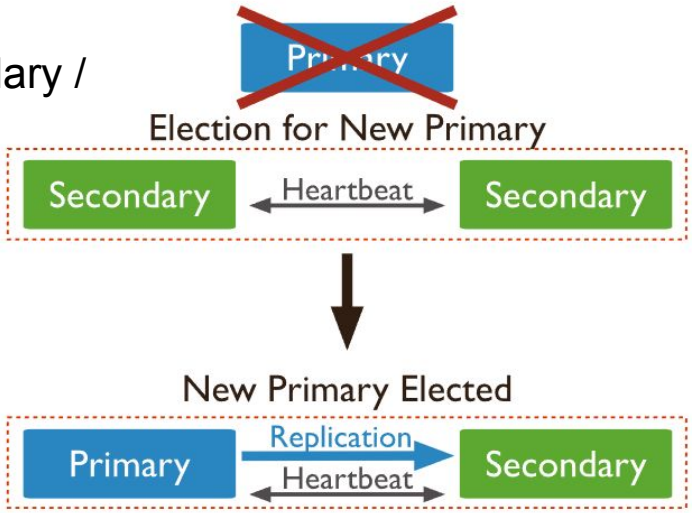
Pokretanje arbitra

```
7
8  // Dodavanje arbitra
9  rs.addArb("localhost:27021")
10
```

```
    {
      _id: 3,
      name: 'localhost:27021',
      health: 1,
      state: 7,
      stateStr: 'ARBITER',
      uptime: 5,
      lastHeartbeat: ISODate('2026-04-19T19:11:42.887Z'),
      lastHeartbeatRecv: ISODate('2026-04-19T19:11:42.890Z'),
      pingMs: Long('0'),
      lastHeartbeatMessage: '',
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: '',
      configVersion: 5,
      configTerm: 3
    }
  ],
  ok: 1
```

Automatski failover

- **Heartbeat** svake 2 sekunde; ako primarni ne odgovori za 10s → pokretanje izbora
- Modifikovani **Raft algoritam**: kandidat traži glasove → pobeđuje ko ima najažurniji oplog i apsolutnu većinu
- Tipično vreme failover-a: **10–30** sekundi
- Read Preference: primary / primaryPreferred / secondary / secondaryPreferred / nearest



Preuzimanje uloge primarnog replica set-a

Inicijalno sekundarni čvor sada postaje primarni

```
},
{
  _id: 2,
  name: 'localhost:27018',
  health: 1,
  state: 1,
  stateStr: 'PRIMARY',
  uptime: 2090,
  optime: { ts: Timestamp({ t: 1776627598, i: 1 }), t: Long('4') },
  optimeDurable: { ts: Timestamp({ t: 1776627598, i: 1 }), t: Long('4') },
  optimeWritten: { ts: Timestamp({ t: 1776627598, i: 1 }), t: Long('4') },
  optimeDate: ISODate('2026-04-19T19:39:58.000Z'),
  optimeDurableDate: ISODate('2026-04-19T19:39:58.000Z'),
  optimeWrittenDate: ISODate('2026-04-19T19:39:58.000Z'),
  lastAppliedWallTime: ISODate('2026-04-19T19:39:58.507Z'),
  lastDurableWallTime: ISODate('2026-04-19T19:39:58.507Z'),
  lastWrittenWallTime: ISODate('2026-04-19T19:39:58.507Z'),
  lastHeartbeat: ISODate('2026-04-19T19:40:04.987Z'),
  lastHeartbeatRecv: ISODate('2026-04-19T19:40:03.344Z'),
  pingMs: Long('0'),
  lastHeartbeatMessage: '',
  syncSourceHost: '',
  syncSourceId: -1,
  infoMessage: '',
  electionTime: Timestamp({ t: 1776627396, i: 1 }),
  electionDate: ISODate('2026-04-19T19:36:36.000Z'),
  configVersion: 9,
  configTerm: 4
},
{
```

```
},
members: [
  {
    _id: 0,
    name: '127.0.0.1:27017',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 240747,
    optime: { ts: Timestamp({ t: 1776627598, i: 1 }), t: Long('4') },
    optimeDate: ISODate('2026-04-19T19:39:58.000Z'),
    optimeWritten: { ts: Timestamp({ t: 1776627598, i: 1 }), t: Long('4') },
    optimeWrittenDate: ISODate('2026-04-19T19:39:58.000Z'),
    lastAppliedWallTime: ISODate('2026-04-19T19:39:58.507Z'),
    lastDurableWallTime: ISODate('2026-04-19T19:39:58.507Z'),
    lastWrittenWallTime: ISODate('2026-04-19T19:39:58.507Z'),
    syncSourceHost: 'localhost:27018',
    syncSourceId: 2,
    infoMessage: '',
    configVersion: 9,
    configTerm: 4,
    self: true,
    lastHeartbeatMessage: ''
  },
  {
```

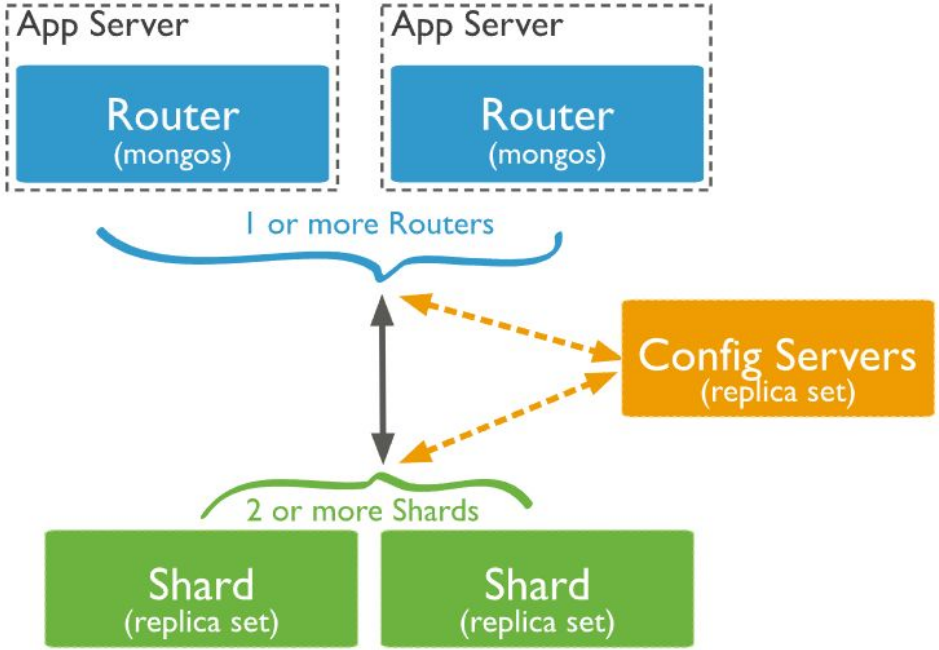
Prethodno primarni čvor nakon oporavka postaje sekundarni

Shardovani klasteri

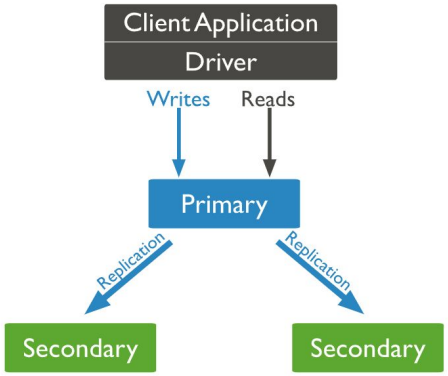
- **Horizontalna skalabilnost**: podaci se distribuiraju na više **shard**-ova (svaki shard se implementira kao replica set)
- Komponente:
  - **Shard** - čuva podskup podataka
  - **Config Server** (CSRS, 3 člana) - metapodaci o chunk-ovima i shardovima
  - **mongos** - query router, može biti više instanci za HA
- **Shard ključ** - visoka kardinalnost, niska učestalost, bez monotone vrednosti
- **Heširani shard ključ** → ravnomerna distribucija pisanja
- **Balancer** - automatski pomera chunk-ove
- Targeted upiti (sadrže shard ključ) - efikasni; **Broadcast** (scatter-gather) - skupi, treba minimizovati

```
}
[direct: mongos] testdb> sh.getBalancerState()
true
[direct: mongos] testdb> sh.isBalancerRunning()
{
  mode: 'full',
  inBalancerRound: false,
  numBalancerRounds: Long('86'),
  term: Long('1'),
  ok: 1,
```

Arhitektura shard-ova



Backup i strategije oporavka



- **Replikacija** ne zamenjuje backup - štiti od kvarova hardvera, ne od slučajnog brisanja
- **mongodump** / **mongorestore** - logički backup (BSON); pogodan za manje baze
- **Point-in-Time Recovery** (PITR): periodični filesystem snapshot + čuvanje oploga
- Odloženi sekundarni čvor (secondaryDelaySecs) - "vremenska mašina"; zaštita od logičkih grešaka; daje prozor za reakciju

```
2026-04-19T23:05:58.790+0200    done dumping `test-db.istorija` (0 documents)
2026-04-19T23:05:58.790+0200    writing `test-db.logovi` to `C:\backup\2026-02-01\test-db\logovi.bson`
2026-04-19T23:05:58.790+0200    done dumping `test-db.transakcije` (0 documents)
2026-04-19T23:05:58.790+0200    writing `banka.narudzbine` to `C:\backup\2026-02-01\banka\narudzbine.bson`
2026-04-19T23:05:58.790+0200    done dumping `test-db.sesije` (0 documents)
2026-04-19T23:05:58.791+0200    done dumping `banka.sesije` (0 documents)
2026-04-19T23:05:58.791+0200    done dumping `banka.narudzbine` (0 documents)
2026-04-19T23:05:58.791+0200    done dumping `test-db.logovi` (0 documents)
```

Dodavanje prvog shard-a i punjenje podacima

```
}
[direct: mongos] testdb> db.users.getShardDistribution()
Shard rs1 at rs1/localhost:27040
{
  data: '265B',
  docs: 5,
  chunks: 1,
  'estimated data per chunk': '265B',
  'estimated docs per chunk': 5
}
---
Totals
{
  data: '265B',
  docs: 5,
  chunks: 1,
  'Shard rs1': [
    '100 % data',
    '100 % docs in cluster',
    '53B avg obj size on shard'
  ]
}
```

Dodavanje drugog shard-a

```
]
[direct: mongos] testdb> db.adminCommand({ listShards: 1 })
{
  shards: [
    {
      _id: 'rs1',
      host: 'rs1/localhost:27040',
      state: 1,
      topologyTime: Timestamp({ t: 1776631116, i: 32 }),
      replSetConfigVersion: Long('-1')
    },
    {
      _id: 'rs2',
      host: 'rs2/localhost:27041',
      state: 1,
      topologyTime: Timestamp({ t: 1776631461, i: 32 }),
      replSetConfigVersion: Long('-1')
    }
  ],
```

Distribucija podataka nakon ukljucivanja drugog shard-a

```
[direct: mongos] testdb> db.users.getShardDistribution()
Shard rs1 at rs1/localhost:27040
{
  data: '45KiB',
  docs: 1035,
  chunks: 1,
  'estimated data per chunk': '45KiB',
  'estimated docs per chunk': 1035
}
---
Shard rs2 at rs2/localhost:27041
{
  data: '45KiB',
  docs: 519,
  chunks: 1,
  'estimated data per chunk': '45KiB',
  'estimated docs per chunk': 519
}
---
Totals
{
  data: '91KiB',
  docs: 1554,
  chunks: 2,
  'Shard rs1': [
    '49.91 % data',
    '66.6 % docs in cluster',
    '90B avg obj size on shard'
  ],
  'Shard rs2': [
    '50.08 % data',
    '33.39 % docs in cluster',
    '90B avg obj size on shard'
  ]
}
```

Geografski distribuirani deployment

- Replica set rasprostranjen na više data centara - zaštita od prirodnih katastrofa
- Asimetrična konfiguracija: **2+1 čvor** → data centar A može preuzeti bez data centra B, ali ne obrnuto
- Simetricna konfiguracija: **1+1 + arbitar** na trećoj lokaciji → oba data centra mogu formirati **kvorum** bez onog drugog
- **Zonal tagging** - aplikacije čitaju sa lokalnog sekundarnog čvora → manja latencija; pisanje uvek na primarnom

```
rs0 [direct: primary] test> cfg = rs.conf()
... cfg.members.forEach(m => print(m._id, m.host, JSON.stringify(m.tags), "priority:", m.priority, "delay:", m.secondaryDe
laySecs))
0 127.0.0.1:27017 {"dc":"us-east","rack":"rack1"} priority: 1 delay: Long('0')
1 localhost:27020 {"dc":"us-west","rack":"rack1"} priority: 1 delay: Long('0')
2 localhost:27018 {"dc":"eu-central","rack":"rack1"} priority: 0 delay: Long('120')
3 localhost:27021 {} priority: 0 delay: Long('0')
```